Module Code & Module Title:

CS4001NT Programming

Assessment Weightage & Type:

10% Individual Coursework

Year and Semester:

2024 Autumn

Student Name: Manoj Katuwal

London Met ID: 24045922

College ID: np05cp4a240306@iic.edu.np

Assignment Due Date: Friday, 2 May 2025

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Tables of contents

Contents

1. Introduction 1
2. Object oriented programing concepts. 1
3. Inheritance 2
4. Encapsulation and Access modifier 7
    1. Private access modifier 7
    2. Public access modifier 8
    3. Protected access modifier 9
5. Abstraction and Polymorphism 10
6. Exception Handling 13
7. Designing a Library Management System 17
8. Course completion conclusion 21
9. Conclusion 24
10. Appendix 24

Table of figures

Figure 1: Parent class 3
Figure 2: Child class 4
Figure 3: main class 4
Figure 4: Output of the program inheritancce 5
Figure 5: Interface example class animal 6
Figure 6: Interface example class Dog 6
Figure 7: main class of an interface example 6
Figure 8:Private access modifier 8
Figure 9:Protected access modifier 10
Figure 10:abstract example1 11
Figure 11:Abstraction example2 11
Figure 12:Abstraction example 3 12
Figure 13:Polymorphism real-word example 13
Figure 14:Exception handling example 14
Figure 15:Custom Exception 15
Figure 16: Contact class with Exception Handling 16
Figure 17:Main class to demonstrate the exception handling 17
Figure 18:abstract class of library item 19
Figure 19:Book class inherits the library item 19
Figure 20:abstract class person 20
Figure 21:Person inherits member 20
Figure 22:Librarian extends person 21
Figure 23: Course Completion Bridge 22
Figure 24: Parent class of my project 23
Figure 25:child class of my project 23

javalearn

Islington College,Nepal

Document Details

Submission ID
trn:oid:::3618:93947012

Submission Date
May 2, 2025, 9:05 AM GMT+5:45

Download Date
May 2, 2025, 9:07 AM GMT+5:45

File Name
javalearn

File Size
11.9 KB

14 Pages

1,922 Words

10,124 Characters

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **10** Not Cited or Quoted 6%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

3% Internet sources
0% Publications
3% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **10** Not Cited or Quoted 6%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

3% Internet sources
0% Publications
3% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
ats.coxenterprises.com		3%
2	Submitted works	
Swinburne University of Technology on 2020-10-30		1%
3	Submitted works	
INTI Universal Holdings SDM BHD on 2024-07-03		<1%
4	Submitted works	
Asia Pacific University College of Technology and Innovation (UCTI) on 2022-05-27		<1%
5	Submitted works	
CTI Education Group on 2016-09-11		<1%
6	Submitted works	
Duquesne University on 2025-05-01		<1%
7	Submitted works	
Swinburne University of Technology on 2018-05-27		<1%

1. Introduction

This report is prepared as a part of programming coursework which was as considered as Independent Learning section. The purpose of this report is to improve my understanding level about java class, interface, OOPs principal etc. In this report I have solve some questions answer and provide some screenshots as using as BLUE J software tool as to write the code which make the answer real use in the program. As the content is self-learning course from the LinkedIn Learning course which I complete independent. As the achievement for the completing this course has given me a certificate as well which I have provided at last. The course helps me to improve my concept regarding the class, OOPs principals, methods, functions which helped me to apply in real-word program scenario as well.

2. Object oriented programing concepts.

a) What are the four main principles of Object-Oriented Programming Language (OOP). Provide a brief explanation.

Answer:

The four main principal of OOP are as follows:

i) Inheritance: Inheritance is the one major key concept of Object-Oriented Programming Language where the subclass or child class inherits some functionality like method, function of their super class or parent class. Inheritance helps the code for readability easily and for not repeating the code as well.

ii) Abstraction: Abstraction is the process of hiding unnecessary information from the user. It is a type of class where we don't create object. Method defining using abstract Keyword must not have body.

iii) Polymorphism: The ability for an object or function to take many different forms depending on the context and situation, the form may be different making your code more flexible and reduce complexity.

iv) Encapsulation: The process of wrapping variables (data) and methods (code) together as a single unit access using access modifier. Public, Private and Protected these modify control the visibility of data

b) How does the concept of class differ from that of an object in java? Provide examples from this course to illustrate your answer.

Answer:

A class is a blueprint that defines the structure and behaviour of an object. A blueprint contains a set of behaviour and attributes that defines an object. An object is an instance of class. Anything which is object in java has its own attributes and functions. Example of this code is given in appendix section.

3. Inheritance

a) Describe how you implemented inheritance on one of the examples provided in the course. How did inheritance promote code reuse.

Answer:

Inheritance is one of the core concept in Object-Oriented Programming Language where the subclass inherits some properties and method of the super class or parent class. In this course I used inheritance in Employee and salesman. Here a brief explanation in this course how I use inheritance in this course.

- Parent class (Employee): This class has basic properties like id, name and salary.
- Child class (Salesman): This class inherits the properties of Parent class and put extra functionality like display function info method.

This way the Salesman class does not define the properties like id, name and salary because it inherits the properties from the employee. By this way the inheritance reduce the code reuse as we don't want to write the properties or attributes of the parent class as it automatically inherits.

For example:

```
// Parent class
public class Employee {
    int id;
    String name;
    double salary;

    // Constructor
    public Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    // Method to display employee details
    public void displayInfo() {
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Salary: $" + salary);
    }
}
```

Figure 1: Parent class

Class

```java
public class Salesman extends Employee {
    double commission;

    // Constructor
    public Salesman(int id, String name, double salary, double commission) {
        super(id, name, salary); // Call the constructor of Employee
        this.commission = commission;
    }

    // Method to display full info
    public void displaySalesmanInfo() {
        super.displayInfo(); // Reuse method from parent class
        System.out.println("Commission: $" + commission);
    }
```

Figure 2: Child class

```java
// Main class to test
public class Main {
    public static void main(String[] args) {
        Salesman s1 = new Salesman(101, "Manoj Katwal", 30000.0, 5000.0);
        s1.displaySalesmanInfo();
    }
}
```

Figure 3: main class

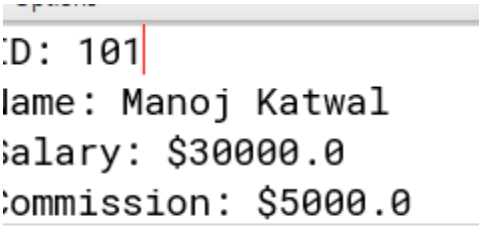

Figure 4: Output of the program inheritancce

b) How did the course demonstrate the use of interface in Java. Provide of example of an interface you created during this course and explain its purpose.

Answer:

In java interface is a contract that defines a set of methos that a class must implement.

We can also that interface is a set of abstract method you would want your class to implement. The method are by default public and abstract.

In this course, we worked an interface to modal of an animal and a dog. The interface animal class a defines a method name makesound. And the different class like dog implement the method in the dog class.

For example:

```
// Interface definition
interface Animal {
    void makeSound(); // abstract method
}
```

Figure 5: Interface example class animal

```
// Class implementing interface
class Dog implements Animal {
    public void makeSound() {
        System.out.println("Dog barks");
    }
}
```

Figure 6: Interface example class Dog

```
// Test class
public class mainbody {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.makeSound(); // Output: Dog barks
    }
}
```

Figure 7: main class of an interface example

4. Encapsulation and Access modifier

a) Explain how encapsulation is achieved in Java. Why is encapsulation is important in Software Development.

Answer:

Encapsulation is achieved in Java by making a class public, private and protected and making public accessor and mutator method. Encapsulation is very important in Software Development because of the following reasons:

- Data security: We can secure a data from hiding and make more securable form encapsulation.
- Controlled access of variable: We can controlled how variables can be accessed or updated from encapsulation.
- Increased Flexibility: Get methods can read-only. For both read and write we use both get and set methods.
- Code reusability: It enhances code reusability and flexibility and making program easier to test and operate.

b) In this course, you worked with access modifier like public, private and protected. Describe with scenario where each would be appropriate used.

Answer:

Java uses access modifier for the accessible of the method, function and attributes within the whole program using public, private and protected access modifier. During the course I Have learned to use the access modifier in a real-word scenario. Below are explanations and example of access modifier.

1. Private access modifier

Definition:

This private modifier helps the program to restrict the attributes and method to directly access from outside the class

Scenario:

While creating gym management system program, where the gym member is the parent class and regular member and premium member are the sub class where I want premium member variables like personal trainer, is full payment, paid amount and discount amount should be private so that the attributes cannot be changed directly and cannot be accessible from outside the class.

For example:

```
public class PremiumMember extends GymMember {
    private final double premiumCharge = 50000;
    private String personalTrainer;
    private boolean isFullPayment;
    private double paidAmount;
    private double discountAmount;
```

Figure 8:Private access modifier

2. Public access modifier

Definition:

The public modifier is the modifier used when we want to give the access of attributes and method in whole program.

Scenario:

While creating a gym management system where you want to give the access of variables and methods to whole program you can use public access modifier.

```java
public class GymUtility {
    public static void displayPlans() {
        System.out.println("Basic Plan, Premium Plan, Student Plan");
    }
}
```

3. Protected access modifier

Definition:

Protected access modifier allows access within the same program or within the same package.

Scenario:

If you have parent class Gym member with an attributes name, id, location and you want this to be directly access by its subclasses like Regular member or Premium Member you can use protected access modifier.

For example:

```
/
blic abstract class GymMember {
    protected int id;
    protected String name;
    protected String location;
    protected String phone;
    protected String email;
    protected String gender;
    protected String DOB;
    protected String membershipStartDate;
    protected int attendance;
    protected double loyaltyPoints;
    protected boolean activeStatus;
```

Figure 9:Protected access modifier

5. Abstraction and Polymorphism

a) How did the course explain the concept of abstraction? Give an example how you applied abstraction in one of your project.

Answer:

Abstraction helps us to hide implementation complexity. This complexity could be form in algorithm, EPI or design. And the main goal of the abstraction is to generalize the feature of given system. The course explained abstraction as a way to focus on what an object does rather then how it does it.

Application Example in my Project (Gym Management System)

Recently I have a made a Java Swing-based Gym Management System. I applied abstraction by creating an abstract class GymMember which defines some common methods to mark attendance of the gym member name markAttendance(). The abstract method does not have any implementation, instead its sub class or child class

Regular member and premium member has its own implementation a it mark its ow attendance on their classes on the basis of some implementation that I implement in that method.

For example:

```
// Abstract Class
abstract class GymMember {
    protected String name;

    public GymMember(String name) {
        this.name = name;
    }

    // Abstract method
    public abstract void markAttendance();
}
```

Figure 10:abstract example1

```
// Subclass RegularMember
class RegularMember extends GymMember {
    public RegularMember(String name) {
        super(name);
    }

    public void markAttendance() {
        System.out.println(name + " has marked attendance for the Regular class.");
    }
}
```

Figure 11:Abstraction example2

```
// Subclass PremiumMember
class PremiumMember extends GymMember {
    public PremiumMember(String name) {
        super(name);
    }

    public void markAttendance() {
        System.out.println(name + " has marked attendance for the Premium class.");
    }
}
```

Figure 12:Abstraction example 3

b) Provide a real-word example where polymorphism would be useful in a Java application. How was polymorphism implemented in one of the course exercise.

Answer:

Polymorphism is an ability for an object or function to take many forms depending on the context and situation, the form may be different making your code more flexible and reducing complexity.

Real-word example:

Imagine a Gym Management System where different types of gym members such as Regular Member and Premium Member have their own type of services or facilities. But all the methods have a same method called discount amount. As regular member also want discount and premium member also want discount in gym. For a Regular Member the discount amount might be fixed and for Premium Member certain discount amount might be fixed on the basis of the duration or something else like that. By using polymorphism we can write the code so that the each gym member type either regular member or premium member get its own type of discount on the basis of certain criteria. Polymorphism allows us to handle all types of discount using same discount amount method with increase code readability and flexibility.

For example:

```
class GymMember {
    public void markAttendance() {
        System.out.println("Attendance marked for general gym member.");
    }
}

class RegularMember extends GymMember {
    @Override
    public void markAttendance() {
        System.out.println("Attendance marked for regular member.");
    }
}

class PremiumMember extends GymMember {
    @Override
    public void markAttendance() {
        System.out.println("Attendance marked for premium member.");
    }
}
```

Figure 13:Polymorphism real-word example

6. Exception Handling

a) Describe how exception handling is implemented in Java. What are the benefits of using exception handling in OOP?

Answer:

Exception handling in Java is a process to handle run-time error where the normal flow of the code is still running. Java provide a powerful model to prevent such exception from this using:

- Try
- Catch
- Finally
- Throw

  For example:

```java
public class ExceptionExample {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;  // This will cause ArithmeticEx
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero.");
        } finally {
            System.out.println("This block always executes.");
        }
    }
}
```

Figure 14:Exception handling example

The benefits of using exception handling inn java are as follows:

- Make readable code
- Make more robust
- Encourage Better Program Design

b) Provide an example of a custom exception class you might create based on a scenario from this course. How would you implement this exception in your code.

Answer:

In java you can create your own custom exceptions by extending the Exception class (for checked exception) or Run-time exception (for unchecked exception).

Scenario from course:

While studying the classes and object in Java from in this course. The teacher give the example of contact class that includes name, phone number and email number as attributes. While user can add their name, phone number and email address. While

user entering the attributes the code can possibly face some custom exception. The following custom-exception are as follows:

- Invalid Phone Number Exception: This exception come when the user put invalid number format or putting character in input or the phone number must be in ten digits.
- Invalid Email Exception: This exception come when user put invalid email format is incorrect.
- Empty Name Exception: This exception come when user put empty name in name format.

The example for this exception are as follows:

```
// Custom exception for empty name
class EmptyNameException extends Exception {
    public EmptyNameException(String message) {
        super(message);
    }
}

// Custom exception for invalid phone number
class InvalidPhoneNumberException extends Exception {
    public InvalidPhoneNumberException(String message) {
        super(message);
    }
}

// Custom exception for invalid email
class InvalidEmailException extends Exception {
    public InvalidEmailException(String message) {
        super(message);
    }
}
```

Figure 15:Custom Exception

```java
public class Contact {
    private String name;
    private String phone;
    private String email;

    public Contact(String name, String phone, String email)
            throws EmptyNameException, InvalidPhoneNumberException, InvalidEmailException {

        if (name == null || name.trim().isEmpty()) {
            throw new EmptyNameException("Name cannot be empty.");
        }

        if (!phone.matches("\\d{10}")) {
            throw new InvalidPhoneNumberException("Phone number must be exactly 10 digits.");
        }

        if (!email.contains("@") || !email.endsWith(".com")) {
            throw new InvalidEmailException("Email must contain '@' and end with '.com'.");
        }

        this.name = name;
        this.phone = phone;
        this.email = email;
    }

    public void display() {
        System.out.println("Name: " + name);
        System.out.println("Phone: " + phone);
        System.out.println("Email: " + email);
    }
}
```

Figure 16: Contact class with Exception Handling

```java
public class Mainclass {
    public static void main(String[] args) {
        try {
            Contact contact1 = new Contact("Manoj", "9801234567", "manoj@example.com");
            contact1.display();

            // Invalid contact (this will throw exception)
            Contact contact2 = new Contact("", "123abc", "invalidemail");

        } catch (EmptyNameException | InvalidPhoneNumberException | InvalidEmailException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Figure 17:Main class to demonstrate the exception handling

7. Designing a Library Management System

a) How would you approach designing a Java application for a library management system using OOP principal? Outline the classes, objects and interactions you would consider (Identify key components, explain interactions and relationships among classes, consider additional feature, provide sample code snippets.)

Answer:

To design a Java application for a library management system using object-oriented programming principal, first we all need to identify the entity(classes), its attributes and relationship between them.

Key OOP principal applied:

a) Encapsulation: Each class will manage its own data using private attributes and public accessor or mutator method.
b) Inheritance: Common properties like id, name can be inherited form a parent class.
c) Abstraction: We can define abstract class to the different section of the books to manage the books and magazines.
d) Polymorphism: Borrowing books can be customized in a different class.

Key classes and their responsibilities:

Class Name	Description
library	Manages all the data likes books, members and borrowing item.
person	Person can be abstract class where the id name can be different.
librarian	It can be inherited from person, can add and remove the books, and give to borrow the books to the members.
Member	It can also be inherited from person can borrow the books, return the books and read the books.
Library Item	It can be the abstract class which contain all the books and necessary all the things in the library.
Book	It can be inherited from the library item differ from the author and it from batch number.
Borrow	Keeps records of the borrowing books form the members including return date, borrowing book.

The example of the program is:

```java
public abstract class LibraryItem {
    private String itemId;
    private String title;

    public LibraryItem(String itemId, String title) {
        this.itemId = itemId;
        this.title = title;
    }

    public abstract void displayDetails();
}
```

Figure 18:abstract class of library item

```java
public class Book extends LibraryItem {
    private String author;

    public Book(String itemId, String title, String author) {
        super(itemId, title);
        this.author = author;
    }

    @Override
    public void displayDetails() {
        System.out.println("Book: " + title + " by " + author);
    }
}
```

Figure 19:Book class inherits the library item

```java
public abstract class Person {
    protected String name;
    protected String id;

    public Person(String name, String id) {
        this.name = name;
        this.id = id;
    }
}
```

Figure 20:abstract class person

```java
public class Member extends Person {
    public Member(String name, String id) {
        super(name, id);
    }

    public void borrowBook(Book book) {
        System.out.println(name + " borrowed: " + book);
    }
}
```

Figure 21:Person inherits member

```java
public class Librarian extends Person {
    public Librarian(String name, String id) {
        super(name, id);
    }

    public void addBook(Book book) {
        System.out.println(name + " added book: " + book);
    }
}
```

Figure 22:Librarian extends person

8. Course completion conclusion

a) Upon completing the Linkedln learning course on Java object-oriented programming you should have earned a course completion bridge. Attach a screenshot or digital copy of your course completion bridge that displays your name and the date of your completion. In addition briefly describe a key concept or project from this course that you found most insightful. How did the course help you enhance your understanding of Object-Oriented programming? Provide specific examples or code snippets to illustrate your point.

Answer:

Upon completing the Linkedln Learning Course on Java Object-Oriented Programming. I earned the following the course completion bridge:

Figure 23: Course Completion Bridge

Key concept I learned:

One of the most insightful concept that I learned from this course was Inheritance. This course teach me to how to reduce the code form inheritance and how to implement parent class properties into child class.

Example from my Project:

```java
public class GymMember {
    protected String name;
    protected int id;

    public GymMember(String name, int id) {
        this.name = name;
        this.id = id;
    }

    public void markAttendance() {
        System.out.println(name + " has attended the gym today.");
    }
}
```

Figure 24: Parent class of my project

```java
public class PremiumMember extends GymMember {
    private boolean hasPersonalTrainer;

    public PremiumMember(String name, int id, boolean hasPersonalTrainer) {
        super(name, id);
        this.hasPersonalTrainer = hasPersonalTrainer;
    }

    @Override
    public void markAttendance() {
        System.out.println(name + " (Premium Member) marked attendance with personal trainer support.");
    }
}
```

Figure 25:child class of my project

This course helped me in following things:

- This course helped me to improve my understanding level between the OOPs concept.
- Designing real-word application using Oops concept.
- I feel more confident to develop object-oriented program after completing this course.

9. Conclusion

Throughout this coursework, I have a sold understanding on the core principals of OOPs such as encapsulation, inheritance, abstraction and polymorphism. By applying the concept of OOPs I have managed to develop ma real-word scenario Gym Management System. I also learned the access modifier, implement interface and exception handling.

However after completing Linkedln Learning course build my confidence to build a real-word program.

10. Appendix

3a) // Parent class

```
public class Employee {
    int id;
    String name;
    double salary;

    // Constructor
    public Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
```

```
        this.salary = salary;
    }

    // Method to display employee details
    public void displayInfo() {
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Salary: $" + salary);
    }
}
// Child class
public class Salesman extends Employee {
    double commission;

    // Constructor
    public Salesman(int id, String name, double salary, double commission) {
        super(id, name, salary); // Call the constructor of Employee
        this.commission = commission;
    }

    // Method to display full info
    public void displaySalesmanInfo() {
        super.displayInfo(); // Reuse method from parent class
```

```
        System.out.println("Commission: $" + commission);
    }
}
// Main class to test
public class Main {
    public static void main(String[] args) {
        Salesman s1 = new Salesman(101, "Manoj Katwal", 30000.0, 5000.0);
        s1.displaySalesmanInfo();
    }
}
```

3b)

```
interface Animal {
    void makeSound(); // abstract method
}

// Class implementing interface
class Dog implements Animal {
    public void makeSound() {
        System.out.println("Dog barks");
    }
}
```

```
// Test class
public class mainbody {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.makeSound(); // Output: Dog barks
    }
}
```

4b)

```
public abstract class GymMember {
    protected int id;
    protected String name;
    protected String location;
    protected String phone;
    protected String email;
    protected String gender;
    protected String DOB;
    protected String membershipStartDate;
    protected int attendance;
    protected double loyaltyPoints;
    protected boolean activeStatus;
}
```

5a)

```
// Parent class
public class GymMember {
    protected String name;
    protected int id;

    public GymMember(String name, int id) {
        this.name = name;
        this.id = id;
    }

    public void markAttendance() {
        System.out.println(name + " has attended the gym today.");
    }
}
// Subclass RegularMember
class RegularMember extends GymMember {
    public RegularMember(String name) {
        super(name);
    }

    public void markAttendance() {
        System.out.println(name + " has marked attendance for the Regular class.");
    }
```

```
}
public class PremiumMember extends GymMember {
    private boolean hasPersonalTrainer;

    public PremiumMember(String name, int id, boolean hasPersonalTrainer) {
        super(name, id);
        this.hasPersonalTrainer = hasPersonalTrainer;
    }

    @Override
    public void markAttendance() {
        System.out.println(name + " (Premium Member) marked attendance with personal trainer support.");
    }
}
```

6a)

```
// Custom exception for empty name
class EmptyNameException extends Exception {
    public EmptyNameException(String message) {
        super(message);
    }
}
```

```
// Custom exception for invalid phone number
class InvalidPhoneNumberException extends Exception {
    public InvalidPhoneNumberException(String message) {
        super(message);
    }
}

// Custom exception for invalid email
class InvalidEmailException extends Exception {
    public InvalidEmailException(String message) {
        super(message);
    }
}
public class Contact {
    private String name;
    private String phone;
    private String email;

    public Contact(String name, String phone, String email)
            throws EmptyNameException, InvalidPhoneNumberException,
InvalidEmailException {

        if (name == null || name.trim().isEmpty()) {
```

```
            throw new EmptyNameException("Name cannot be empty.");
        }

        if (!phone.matches("\\d{10}")) {
            throw new InvalidPhoneNumberException("Phone number must be exactly 10 digits.");
        }

        if (!email.contains("@") || !email.endsWith(".com")) {
            throw new InvalidEmailException("Email must contain '@' and end with '.com'.");
        }

        this.name = name;
        this.phone = phone;
        this.email = email;
    }

    public void display() {
        System.out.println("Name: " + name);
        System.out.println("Phone: " + phone);
        System.out.println("Email: " + email);
    }
}
```

```
public class Mainclass {
    public static void main(String[] args) {
        try {
            Contact contact1 = new Contact("Manoj", "9801234567", "manoj@example.com");
            contact1.display();

            // Invalid contact (this will throw exception)
            Contact contact2 = new Contact("", "123abc", "invalidemail");

        } catch (EmptyNameException | InvalidPhoneNumberException | InvalidEmailException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

7a)

```
public abstract class LibraryItem {
    private String itemId;
    public String title;

    public LibraryItem(String itemId, String title) {
        this.itemId = itemId;
```

```
        this.title = title;
    }

    public abstract void displayDetails();
}
public abstract class Person {
    protected String name;
    protected String id;

    public Person(String name, String id) {
        this.name = name;
        this.id = id;
    }
}
public class Librarian extends Person {
    public Librarian(String name, String id) {
        super(name, id);
    }

    public void addBook(Book book) {
        System.out.println(name + " added book: " + book);
    }
}
```

```
public class Book extends LibraryItem {
    private String author;

    public Book(String itemId, String title, String author) {
        super(itemId, title);
        this.author = author;
    }

    @Override
    public void displayDetails() {
        System.out.println("Book: " + title + " by " + author);
    }
}
public class Member extends Person {
    public Member(String name, String id) {
        super(name, id);
    }

    public void borrowBook(Book book) {
        System.out.println(name + " borrowed: " + book);
    }
}
```

8)

```
// Parent class
public class GymMember {
    protected String name;
    public int id;

    public GymMember(String name, int id) {
        this.name = name;
        this.id = id;
    }

    public void markAttendance() {
        System.out.println(name + " has attended the gym today.");
    }
}

// Child class
public class PremiumMember extends GymMember {
    private boolean hasPersonalTrainer;

    public PremiumMember(String name, int id, boolean hasPersonalTrainer) {
        super(name, id);
        this.hasPersonalTrainer = hasPersonalTrainer;
    }
```

```
    @Override
    public void markAttendance() {
        System.out.println(name + " (Premium Member) marked attendance with personal trainer support.");
    }
}
```